Sure! The following sections break down the steps for **epochs 15, 20, 25, and 30**. They follow the same structure as the epoch 10 workflow, with adjustments to save results accordingly for each epoch set.

---

## Section 1: Data Loading and Preprocessing for Fruit Classification (for all epoch sets)

1. **Load the Dataset**:
    - This section remains the same for all epoch sets as it focuses on loading the dataset (`labels (1).csv`).

```
df = pd.read_csv("labels (1).csv")
```

2.
3. **Data Augmentation**:
    - Same as in the previous section, apply augmentation to training data for all epoch sets.

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
validation_datagen = ImageDataGenerator(rescale=1./255)
```

4.
5. **Create Data Generators**:
    - Create the same data generators (`train_fruit_generator`, `val_fruit_generator`, `test_fruit_generator`) for all epoch sets.

```
train_fruit_generator = train_datagen.flow_from_dataframe(...)
```

```
val_fruit_generator = validation_datagen.flow_from_dataframe(...)

test_fruit_generator = validation_datagen.flow_from_dataframe(...)
```

6.

---

## Section 2: Model Creation, Compilation, and Training for Fruit Classification

1. **Create the MobileNetV2 Model**:
    - The model architecture will stay the same across all epoch sets. It uses MobileNetV2 with custom layers for fruit classification.

```
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False
model = models.Sequential([...])  # Add custom layers
```

2.
3. **Compile the Model**:
    - Compile the model with the same optimizer and metrics.

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy',
'Precision', 'Recall', 'AUC'])
```

4.
5. **Train the Model for Different Epochs (15, 20, 25, 30)**:
    - Train the model for each epoch set and save the results accordingly.

**For Epoch 15:**

```
 history_15 = model.fit(...)

# Save the model and history for Epoch 15
model.save('epoch_15/model/fruit_classification_model_epoch_15.h5')
history_15_df = pd.DataFrame(history_15.history)
history_15_df.to_csv('epoch_15/history/fruit_classification_history_epoch_15.csv', index=False)
```

**For Epoch 20:**

```
history_20 = model.fit(...)

# Save the model and history for Epoch 20
model.save('epoch_20/model/fruit_classification_model_epoch_20.h5')
history_20_df = pd.DataFrame(history_20.history)
history_20_df.to_csv('epoch_20/history/fruit_classification_history_epoch_20.csv', index=False)
```

**For Epoch 25:**

```
history_25 = model.fit(...)

# Save the model and history for Epoch 25
model.save('epoch_25/model/fruit_classification_model_epoch_25.h5')
history_25_df = pd.DataFrame(history_25.history)
history_25_df.to_csv('epoch_25/history/fruit_classification_history_epoch_25.csv', index=False)
```

**For Epoch 30:**

```
history_30 = model.fit(...)

# Save the model and history for Epoch 30
model.save('epoch_30/model/fruit_classification_model_epoch_30.h5')
history_30_df = pd.DataFrame(history_30.history)
history_30_df.to_csv('epoch_30/history/fruit_classification_history_epoch_30.csv', index=False)
```

6.

---

## Section 3: Model Evaluation and Analysis for Fruit Classification (for all epoch sets)

1. **Evaluate the Model on the Test Set**:
    - Predict the test set and evaluate the results for each epoch set.

**For Epoch 15**:

```
test_preds_15 = model.predict(test_fruit_generator)
test_preds_classes_15 = test_preds_15.argmax(axis=-1)

test_class_report_15 = classification_report(test_fruit_generator.classes,
test_preds_classes_15, target_names=train_fruit_generator.class_indices.keys())
with open('epoch_15/fruit_classification_report_epoch_15.txt', 'w') as f:
```

```
    f.write(test_class_report_15)
```
**For Epoch 20**:

```
 test_preds_20 = model.predict(test_fruit_generator)
test_preds_classes_20 = test_preds_20.argmax(axis=-1)

test_class_report_20 = classification_report(test_fruit_generator.classes,
test_preds_classes_20, target_names=train_fruit_generator.class_indices.keys())
with open('epoch_20/fruit_classification_report_epoch_20.txt', 'w') as f:
    f.write(test_class_report_20)
```
**For Epoch 25**:

```
 test_preds_25 = model.predict(test_fruit_generator)
test_preds_classes_25 = test_preds_25.argmax(axis=-1)

test_class_report_25 = classification_report(test_fruit_generator.classes,
test_preds_classes_25, target_names=train_fruit_generator.class_indices.keys())
with open('epoch_25/fruit_classification_report_epoch_25.txt', 'w') as f:
    f.write(test_class_report_25)
```
**For Epoch 30**:

```
 test_preds_30 = model.predict(test_fruit_generator)
test_preds_classes_30 = test_preds_30.argmax(axis=-1)

test_class_report_30 = classification_report(test_fruit_generator.classes,
test_preds_classes_30, target_names=train_fruit_generator.class_indices.keys())
with open('epoch_30/fruit_classification_report_epoch_30.txt', 'w') as f:
    f.write(test_class_report_30)
```

2. 
3. **Confusion Matrix**:

    - Generate and save confusion matrices for each epoch set.

**For Epoch 15**:

```
 cm_15 = confusion_matrix(test_fruit_generator.classes, test_preds_classes_15)
cm_df_15 = pd.DataFrame(cm_15, index=train_fruit_generator.class_indices.keys(),
columns=train_fruit_generator.class_indices.keys())
sns.heatmap(cm_df_15, annot=True, fmt='d', cmap='Blues')
plt.savefig('epoch_15/confusion_matrix_fruit_epoch_15.png')
```
**For Epoch 20**:

```
 cm_20 = confusion_matrix(test_fruit_generator.classes, test_preds_classes_20)
```

```
cm_df_20 = pd.DataFrame(cm_20, index=train_fruit_generator.class_indices.keys(),
columns=train_fruit_generator.class_indices.keys())
sns.heatmap(cm_df_20, annot=True, fmt='d', cmap='Blues')
plt.savefig('epoch_20/confusion_matrix_fruit_epoch_20.png')
```

**For Epoch 25**:

```
cm_25 = confusion_matrix(test_fruit_generator.classes, test_preds_classes_25)
cm_df_25 = pd.DataFrame(cm_25, index=train_fruit_generator.class_indices.keys(),
columns=train_fruit_generator.class_indices.keys())
sns.heatmap(cm_df_25, annot=True, fmt='d', cmap='Blues')
plt.savefig('epoch_25/confusion_matrix_fruit_epoch_25.png')
```

**For Epoch 30**:

```
cm_30 = confusion_matrix(test_fruit_generator.classes, test_preds_classes_30)
cm_df_30 = pd.DataFrame(cm_30, index=train_fruit_generator.class_indices.keys(),
columns=train_fruit_generator.class_indices.keys())
sns.heatmap(cm_df_30, annot=True, fmt='d', cmap='Blues')
plt.savefig('epoch_30/confusion_matrix_fruit_epoch_30.png')
```

4.

---

## Section 4: Evaluation Plots for Fruit Classification (for all epoch sets)

1. **Plot Training and Validation Metrics**:
    - Accuracy, loss, precision, recall, and F1-score vs. epochs for each epoch set.

**For Epoch 15**:

```
# Plot Accuracy vs Epochs
plt.plot(history_15.history['accuracy'], label='Training Accuracy')
plt.plot(history_15.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig('epoch_15/accuracy_vs_epoch_fruit_epoch_15.png')
```

**For Epoch 20**:

```
# Plot Accuracy vs Epochs
plt.plot(history_20.history['accuracy'], label='Training Accuracy')
plt.plot(history_20.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
```

```
plt.ylabel('Accuracy')
plt.legend()
plt.savefig('epoch_20/accuracy_vs_epoch_fruit_epoch_20.png')
```

2.

**For Epoch 25**:

```
# Plot Accuracy vs Epochs
plt.plot(history_25.history['accuracy'], label='Training Accuracy')
plt.plot(history_25.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig('epoch_25/accuracy_vs_epoch_fruit_epoch_25.png')
```

**For Epoch 30**:

```
# Plot Accuracy vs Epochs
plt.plot(history_30.history['accuracy'], label='Training Accuracy')
plt.plot(history_30.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig('epoch_30/accuracy_vs_epoch_fruit_epoch_30.png')
```

---

## Continue with Epoch 15-30 Saving Results:

For each epoch set (15, 20, 25, 30), continue with the same pattern: saving models, training history, evaluation results, confusion matrix, and evaluation plots.